

Django 5 With ML/AI Course**4th Assessment - Final Integrated Machine Test****Question Paper 1: AI-Enabled Recruitment Screening and Interview Intelligence Portal**

Coverage	Day 1 to Day 73 / Full Syllabus
Assessment Type	Single Scenario-Based Machine Test Coding Round
Number of Questions	1 Integrated Question
Total Marks	100
Duration	3 Hours
Viva	Yes

General Instructions

- This is a single-question machine test. All sub-sections are part of the same integrated problem statement, not separate independent questions.
- Use only the topics covered in the syllabus from Day 1 to Day 73. Do not use React, WebSockets, Docker, Kubernetes, cloud services, external LLM APIs, or any topic outside the syllabus.
- Use Python 3 and Django 5. Use Django Templates for the web UI. Use Django REST Framework for APIs.
- Use PostgreSQL for database work. Use Django ORM wherever suitable and write the requested raw SQL queries separately in a file named `sql_queries.sql`.
- Use JWT or Token Authentication for secured API access. Role-based permission must be implemented.
- Use Celery for asynchronous/background jobs and Redis for broker/cache wherever asked.
- Use NumPy, Pandas, Matplotlib, and Scikit-learn for the ML component. Use NLTK for NLP. Use TensorFlow/Keras for the deep-learning component as a minimal working model or clearly executable prototype.
- The project must run locally. The evaluator should be able to create the database, run migrations, start the server, run Celery worker, and test APIs using Postman/Swagger.
- Hard-coded secrets, broken migrations, non-running projects, copied code without understanding, or missing README will be penalized heavily.

Submission Requirements

- Complete Django project source code as a zip file.
- README.md with setup steps, environment variables, migration commands, superuser/admin credentials for testing, Celery/Redis run commands, and API testing instructions.
- Sample CSV/Excel dataset and at least one sample media/image file if the question requires media upload.
- `sql_queries.sql` containing required SELECT, WHERE, AND/OR/LIKE/BETWEEN, LIMIT/ORDER BY, aggregation, GROUP BY/HAVING, join, UNION ALL, and INTERSECT examples relevant to the project.
- A short notes.txt file explaining important assumptions, known limitations, and how the ML/NLP/DL parts were implemented.

Question 1: Integrated Full-Syllabus Machine Test**Business Scenario**

A product company wants to hire Python/Django freshers from multiple campuses. HR receives thousands of candidate records through CSV/Excel files and resume text snippets. The company needs an internal Django portal that can ingest applications, validate candidate data, shortlist candidates using rule-based and ML-based scoring, analyze resume text, schedule interview slots, and expose secured APIs for HR users and interviewers.

Task

Design and implement a working local prototype for the system described above. The solution must cover the following functional and technical requirements. All listed points are part of the same single question and must be implemented as one integrated project.

Candidate data ingestion and Python automation

- Create a Python ingestion utility that accepts a CSV/Excel file containing candidate_name, email, phone, college, applied_role, skills, experience_months, notice_period_days, expected_salary, resume_text, portfolio_url, and historical_selection_status.
- Validate emails and phone numbers using regular expressions. Clean string data using indexing, slicing, removeprefix(), removesuffix(), split(), join(), strip(), and case normalization.
- Use list, tuple, set, frozenset, dictionary, dictionary comprehension, set comprehension, iterators, while loops, break/continue, shallow copy, deep copy, decorators, recursion, *args, **kwargs, lambda, local/global/nonlocal usage, inheritance, operator overloading, and user-defined exceptions wherever they naturally fit.
- Generate a rejected_rows.csv file with reason-wise errors and an accepted_rows.csv file after cleaning.

Django portal requirements

- Create models such as JobRole, Candidate, ApplicationBatch, ScreeningResult, InterviewSlot, and InterviewFeedback.
- Implement Django admin customization with filters, fieldsets, searchable fields, and meaningful list display columns.
- Build function-based views and class-based views for uploading candidate files, listing candidates, viewing candidate details, editing screening status, and deleting invalid records.
- Use Django Template Language with variables, for loops, if/else tags, context, URL reversing, redirects, GET/POST handling, static CSS/JavaScript, media upload, and image/resume file support.
- Use AJAX to check duplicate candidate emails before final form submission and to dynamically load interview slots for a selected role. CSRF handling must be correct.
- Use cookies to remember the last selected role filter and sessions to store the current screening batch selected by the HR user.
- Create a custom middleware that logs request path, request method, processing time, and user role. It should also block non-authenticated access to internal dashboard URLs.

Database, SQL, ORM, and reports

- Use migrations properly and create meaningful relationships between models.
- Create ORM queries using get(), filter(), exclude(), ordering, slicing, and JSON responses for individual and all candidates.
- Write raw PostgreSQL queries for candidate search, role-wise counts, selected/rejected aggregation, salary range filtering with BETWEEN, LIKE search over skills, inner join between candidates and roles, full outer join for role/candidate availability, natural join where suitable, UNION ALL for selected and waitlisted candidates, and INTERSECT for candidates who match both Python and Django skills.
- Create a dashboard summary with COUNT, SUM, MIN, MAX, AVG, GROUP BY, and HAVING.

DRF, authentication, permissions, and verification

- Expose APIs for login, registration, logout, candidate CRUD, batch upload status, screening result list, and interview feedback submission.
- Use serializers for GET/POST/PUT/DELETE, ModelSerializer validation, and at least one HyperLinkedModelSerializer endpoint.
- Implement JWT or Token Authentication with access and refresh token flow. Implement HR, Interviewer, and Admin roles with custom permissions.
- Add email verification or OTP verification for new HR users before they can access secured APIs.
- Expose Swagger documentation and demonstrate at least five API requests through Postman.

Celery, Redis, ML, NLP, and DL

- Process large candidate uploads asynchronously through Celery. Store job status and progress. Use Redis as broker and for caching top shortlisted candidates per role.
- Build an ML pipeline using Pandas, NumPy, Matplotlib, and Scikit-learn to predict candidate shortlist probability from cleaned features. Include missing value handling, duplicate removal, useless column removal, outlier treatment, encoding categorical variables, feature selection, train/test split, and at least two algorithms among Logistic Regression, Random Forest, SVM, Naive Bayes, and KNN.
- Deploy the selected ML model through a secured DRF endpoint that accepts candidate features and returns prediction plus confidence/score.
- Use NLTK to tokenize resume_text, create text vectors, perform POS tagging, and compute skill-keyword match score.
- Create a minimal TensorFlow/Keras ANN model for shortlist classification or resume score classification. It may use the same processed tabular/text-derived features, but it must be executable and explained.

Mandatory Edge Cases

- Invalid file format, missing required columns, empty uploaded file, duplicate rows, invalid numeric values, and invalid email/phone/code patterns must be handled safely.
- Unauthorized users must not access protected pages or APIs. Expired/invalid tokens must return proper errors.
- The system should not crash when optional text/media fields are missing.
- ML/NLP/DL modules must be callable repeatedly without corrupting existing database records.
- Celery task failure must be visible through status/progress records or API responses.

Evaluation Rubric

Evaluation Area	Marks	Expected Evidence
Python core scripting, validation, OOP, decorators, recursion, exceptions, collections, string handling	10	Running scripts, validators, custom exceptions, OOP usage, clean Python implementation.
Django MVT, templates, forms/ModelForms, static/media, admin customization, AJAX, CSRF, cookies, sessions, middleware, CBV	20	Working web UI, admin, forms, templates, AJAX, security handling, middleware, and upload flows.
PostgreSQL, raw SQL, models, migrations, ORM QuerySets, JSON responses	15	Correct schema, migrations, ORM usage, raw SQL file, dashboard analytics, JSON responses.
DRF serializers, API CRUD, validation, Postman/Swagger, JWT/Token auth, roles, custom permissions, email/OTP verification	15	Working secured APIs, serializers, role permissions, token flow, verification, Swagger/Postman proof.
Asynchronous execution, Celery worker, Redis broker/cache, multiprocessing vs multithreading understanding	10	Working worker task, Redis usage, progress/status tracking, cached data where suitable.
Data processing, visualization, feature engineering, ML model training/evaluation/deployment	15	Executable notebook/script or integrated module with preprocessing, training, metrics, and endpoint use.
NLP using NLTK plus TensorFlow/Keras deep-learning component	10	Working NLTK pipeline and minimal Keras model/prototype with explanation.
Code quality, project structure, README, edge-case handling, and viva readiness	5	Readable code, setup documentation, clear assumptions, and ability to explain decisions in viva.

Viva Focus Areas

- Explain why GET and POST were used in different parts of the portal.
- Explain MVT flow from URL to view to template for candidate upload.
- Explain authentication, refresh token, and custom permission logic.
- Explain why Celery was required instead of processing everything inside the HTTP request.
- Explain feature engineering, model choice, and how resume text was converted into numerical features.

Important Restriction

The evaluator will reject any major implementation that depends on topics outside the syllabus. The expected solution should be built using Python, Django, Django Templates, Django Admin, PostgreSQL, Django ORM, DRF, Postman/Swagger, JWT/Token Authentication, email/OTP verification, Celery, Redis, NumPy, Pandas, Matplotlib, Scikit-learn, NLTK, and TensorFlow/Keras only as required by the syllabus.